

# Manual de la Base de Datos - The Route66 Market (Supabase)

Version: 1.0 | Fecha: 10/09/2026  
Plataforma: Supabase (PostgreSQL)

## 1. Introduccion

El proyecto usa **Supabase** como base de datos (PostgreSQL), storage para fotos y autenticacion. Todos los scripts SQL viven en la raiz del proyecto excluidos de GitHub (\*.sql en .gitignore por politica del repositorio): solo se corren localmente en el **SQL Editor** de Supabase.

## 2. Tablas principales

| Tabla           | Proposito                                                                                                     |
|-----------------|---------------------------------------------------------------------------------------------------------------|
| productos       | Catalogo: nombre, marca, precio, categoria, stock, origen, foto, codigo de barras                             |
| pedidos         | Pedidos: cliente, productos (JSON), totales, entrega, estado, fechas                                          |
| inventario      | Movimientos de stock (producto_id, tipo, cantidad)                                                            |
| finanzas        | Ingresos/gastos: tipo, categoria contable, monto, fecha                                                       |
| lealtad         | Programa de lealtad por cliente (productos comprados, regalo)                                                 |
| pagos           | Creditos: adeudo_total, quincenas (totales/pagadas/liquidadas/pendientes), abonos, cargos, morosidad, estados |
| cupones         | Cupones del carrusel de la tienda (codigo, descuento, vigente)                                                |
| cupon_usos      | Usos de cupon por correo (unico por combinacion correo+cupon)                                                 |
| noticias        | Avisos y promociones mostrados en la tienda                                                                   |
| marcas          | Marcas (cards/selects en la tienda y el panel)                                                                |
| lugares_entrega | Puntos de entrega v2: lugar, costo, orden y horario_fijo                                                      |
| settings        | Llave/valor para bases de reinicio de las finanzas                                                            |

Todos los scripts activan **RLS (Row Level Security)** con politicas abiertas para que la app pueda leer/escribir con la **clave anonima** de Supabase.

## 3. Scripts SQL del proyecto

- > Todos son **idempotentes** (se pueden re-ejecutar sin romper nada). Devuelven correctamente en el SQL Editor de Supabase (mensaje verde). **No mezcles** categorias: cada script cumple un proposito distinto.

### 3.1 sql\_completo.sql - Esquema / reparacion COMPLETA

Es el script maestro. Hace lo siguiente:

- Crea o repara tablas: pagos, lugares\_entrega, marcas, cupon\_usos, noticias, finanzas, settings.
- Garantiza las columnas nuevas:
- productos.marca, productos.tienda\_origen, productos.codigo\_barras, productos.imagen\_url.
- pedidos.lugar\_entrega, pedidos.fecha\_vendido, pedidos.fecha\_entregado, pedidos.punto\_entrega, vista relacionada.
- Columnas de creditos en pagos (adeudo total, quincenas, liquidacion).
- Activa RLS con politicas abiertas.
- Siembra los 6 puntos de entrega v2 y las 15 marcas iniciales (solo si las tablas estan vacias).
- Elimina la columna huerfana pagos.nombre.

**Cuando usarlo:** base nueva, o para poner toda la BD "al dia".

Es la base de los demas scripts.

### 3.2 `sql\_lugares\_entrega.sql` - Puntos de entrega definitivos (v2)

- Crea la tabla + columna `horario\_fijo`.
  - Activa RLS.
  - **Reemplaza** el contenido por los 6 puntos definitivos:
  - Fijos (\$0): Apodaca centro 08:00-08:30, San Nicolas centro 09:00-09:30, Costco Escobedo 10:00-10:30.
  - Coordinados (\$200): San Pedro, Monterrey, Guadalupe (horario por WhatsApp).
- Cuando usarlo**: para fijar los puntos de entrega actuales. Ejecutalo ANTES de cargar pedidos vinculados a estos lugares.

### 3.3 `sql\_storage\_productos.sql` - Bucket de fotos

- Crea (idempotente) el bucket publico `productos` en Supabase Storage.
  - Politicas sobre `storage.objects` para el bucket `productos`:  
`select`, `insert`, **update** y **delete** (permite reemplazar y borrar fotos de producto).
  - URL publica de cada foto:  
`https://<tu-proyecto>.supabase.co/storage/v1/object/public/productos/<archivo>`
- Cuando usarlo**: una sola vez, para poder subir/editar/borrar fotos desde el panel.

### 3.4 `sql\_datos\_reales.sql` - LIMPIAR para datos reales

- Elimina TODO el contenido de: `productos`, `pedidos`, `inventario`, `finanzas`, `lealtad`, `pagos`, `cupones`, `noticias`, `cupon\_usos` y `settings`.
  - **NO toca** `lugares\_entrega` ni `marcas` (se conservan).
  - Reinicia `settings` en 0 y limpia la columna/CHECK huérfanos de `pagos`.
- Cuando usarlo**: despues de respaldar, para empezar a capturar **datos reales** desde el panel. No inserta nada.

### 3.5 `sql\_datos\_dummy.sql` - LIMPIAR + datos de ejemplo

- Igual que el de datos reales (borra todo), pero ademas **inserta datos dummy**: 6 productos, 5 pedidos (puntos fijos y coordinados), inventario, finanzas, lealtad, pagos, cupones y noticias.
  - Respeta las fotos ya subidas (las respalda y restaura por `codigo\_barras`).
  - Reinserta marcas sin duplicarlas (indice unico por nombre).
- Cuando usarlo**: para probar el panel y la tienda con datos de muestra.

## 4. Orden recomendado de ejecucion

- 1) sql\_completo.sql (esquema al dia)
- 2) sql\_storage\_productos.sql (bucket de fotos, una vez)
- 3) sql\_lugares\_entrega.sql (puntos v2)
- 4) sql\_datos\_reales.sql o sql\_datos\_dummy.sql (nunca ambos)

No ejecutes `sql\_datos\_reales.sql` y `sql\_datos\_dummy.sql` seguidos: el segundo borraría los datos del primero (ambos vacían).

## 5. Advertencias

- Haz un **\*\*respaldo\*\*** (Export > Database dump o la tabla completa) antes de correr cualquier ``sql_datos_*` (vacía tablas).
- Los scripts de datos arrancan PDF/triggers; si el SQL Editor marca error por algún objeto, **\*\*verifica si esa sección ya existía\*\*** y vuelve a intentar: son idempotentes.
- No restaures el esquema viejo de lugares de entrega en ``sql_completo.sql``: la app usa el **\*\*v2\*\*** (con ``horario_fijo``) y tiene fallbacks en ``js/catalog.js`` y ``js/admin.js``.